```python
import random
import tkinter as tk
from tkinter import messagebox, ttk

class DungeonEscapeApp:
    """A small graphical version of the dungeon escape game."""

    MAX_HEALTH = 60
    ENEMIES = [
        ["Slime", 18, 4],
        ["Goblin", 28, 7],
        ["Skeleton King", 40, 9],
        ["Shadow Dragon", 60, 15],
    ]
    ROOMS = ["Moss Cavern", "Forgotten Cellar", "Royal Crypt", "Dragon Throne"]
    WEAPONS = [["Wooden Sword", 1], ["Iron Sword", 2], ["Dragon Blade", 3]]
    LOOT_POOL = ["Health Potion", "Gold Coin", "Magic Scroll"]

    def __init__(self, root):
        self.root = root
        self.root.title("Dungeon Escape")
        self.root.geometry("900x760")
        self.root.minsize(820, 700)

        self.reset_game_state()

        self.build_ui()
        self.show_start_screen()

    def build_ui(self):
        self.root.configure(bg="#111827")
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TFrame", background="#111827")
        style.configure("Panel.TFrame", background="#1f2937")
        style.configure("Title.TLabel", background="#111827", foreground="#f8fafc", font=("Segoe UI", 28, "bold"))
        style.configure("Subtitle.TLabel", background="#111827", foreground="#94a3b8", font=("Segoe UI", 11))
        style.configure("Panel.TLabel", background="#1f2937", foreground="#e5e7eb", font=("Segoe UI", 11))
        style.configure("Stat.TLabel", background="#1f2937", foreground="#f8fafc", font=("Segoe UI", 16, "bold"))
        style.configure("Action.TButton", font=("Segoe UI", 11, "bold"), padding=10)

        self.container = ttk.Frame(self.root, padding=28)
        self.container.pack(fill="both", expand=True)

        ttk.Label(self.container, text="DUNGEON ESCAPE CLICK", style="Title.TLabel").pack(anchor="w")
        self.subtitle = ttk.Label(self.container, style="Subtitle.TLabel")
        self.subtitle.pack(anchor="w", pady=(2, 22))

        self.start_frame = ttk.Frame(self.container, style="Panel.TFrame", padding=24)
        ttk.Label(self.start_frame, text="ENTER YOUR NAME!!!!!!!", style="Panel.TLabel").pack(anchor="w")
        ttk.Label(self.start_frame, text="But before that here are the instructions; Read carefully! Attack to damage enemies. Heal with a potion, or run to skip a room. Defeat all four encounters, collect loot, and finish with the highest s
        self.name_entry = ttk.Entry(self.start_frame, font=("Segoe UI", 12))
        self.name_entry.pack(fill="x", pady=(8, 18))
        self.name_entry.bind("<Return>", lambda _event: self.start_game())
        ttk.Button(self.start_frame, text="Enter the dungeon", style="Action.TButton", command=self.start_game).pack(fill="x")
```

```python
        self.name_entry.bind("<Return>", lambda _event: self.start_game())
        ttk.Button(self.start_frame, text="Enter the dungeon", style="Action.TButton", command=self.start_game).pack(fill="x")

        self.game_frame = ttk.Frame(self.container)
        self.status_frame = ttk.Frame(self.game_frame, style="Panel.TFrame", padding=18)
        self.status_frame.pack(fill="x")
        self.player_label = ttk.Label(self.status_frame, style="Stat.TLabel")
        self.player_label.pack(side="left")
        self.enemy_label = ttk.Label(self.status_frame, style="Stat.TLabel")
        self.enemy_label.pack(side="right")

        self.enemy_bar = ttk.Progressbar(self.game_frame, maximum=100, mode="determinate")
        self.enemy_bar.pack(fill="x", pady=(14, 18))

        self.scene = tk.Canvas(self.game_frame, height=190, bg="#161d31", highlightthickness=0)
        self.scene.pack(fill="x", pady=(0, 14))

        content = ttk.Frame(self.game_frame)
        content.pack(fill="both", expand=True)
        log_frame = ttk.Frame(content, style="Panel.TFrame", padding=12)
        log_frame.pack(side="left", fill="both", expand=True, padx=(0, 12))
        self.log = tk.Text(log_frame, height=14, state="disabled", wrap="word", bg="#1f2937", fg="#dbeafe", insertbackground="white", relief="flat", font=("Consolas", 10))
        self.log.pack(fill="both", expand=True)

        side = ttk.Frame(content, style="Panel.TFrame", padding=16)
        side.pack(side="right", fill="y")
        ttk.Label(side, text="INVENTORY", style="Panel.TLabel").pack(anchor="w")
        self.inventory_label = ttk.Label(side, style="Panel.TLabel", justify="left")
        self.inventory_label.pack(anchor="w", pady=(8, 24))

        self.action_bar = ttk.Frame(self.game_frame, style="Panel.TFrame", padding=10)
        self.action_bar.pack(fill="x", pady=(10, 0), before=content)
        ttk.Label(self.action_bar, text="ACTIONS", style="Panel.TLabel").pack(side="left", padx=(0, 12))
        self.attack_button = self.create_action_button("Attack", self.attack)
        self.heal_button = self.create_action_button("Heal", self.heal)
        self.run_button = self.create_action_button("Run", self.run_away)
        ttk.Button(self.action_bar, text="Play Again", command=self.show_start_screen).pack(side="left", expand=True, fill="x", padx=4)

    def create_action_button(self, text, command):
        button = ttk.Button(self.action_bar, text=text, style="Action.TButton", command=command)
        button.pack(side="left", expand=True, fill="x", padx=4)
        return button

    def reset_game_state(self):
        self.name = ""
        self.health = self.MAX_HEALTH
        self.enemy_index = 0
        self.enemy_hp = 0
        self.enemy_name = ""
        self.enemy_damage = 0
        self.inventory = []
        self.battle_history = []
        self.loot_history = []
        self.score = 0
        self.gold = 0
```

```
        self.enemy_index = 0
        self.enemy_hp = 0
        self.enemy_name = ""
        self.enemy_damage = 0
        self.inventory = []
        self.battle_history = []
        self.loot_history = []
        self.score = 0
        self.gold = 0
        self.weapon_level = 1
        self.game_active = False

    def show_start_screen(self):
        self.game_active = False
        self.game_frame.pack_forget()
        self.start_frame.pack(fill="x")
        self.subtitle.configure(text="Survive the dungeon. Choose your moves carefully.")
        self.name_entry.focus_set()

    def start_game(self):
        player_name = self.name_entry.get().strip() or "Eliezr EJ A. Maique"
        self.reset_game_state()
        self.name = player_name
        self.inventory = ["Health Potion", "Wooden Sword"]
        self.game_active = True
        self.start_frame.pack_forget()
        self.game_frame.pack(fill="both", expand=True)
        self.log.configure(state="normal")
        self.log.delete("1.0", "end")
        self.log.configure(state="disabled")
        self.write_log(f"{self.name} enters the dungeon with 60 HP!")
        self.begin_battle()

    def begin_battle(self):
        if self.enemy_index >= len(self.ENEMIES):
            self.finish_game("You defeated every enemy and escaped the dungeon!")
            return
        enemy_record = self.ENEMIES[self.enemy_index]
        self.enemy_name = enemy_record[0]
        self.enemy_hp = enemy_record[1]
        self.enemy_damage = enemy_record[2]
        room = self.ROOMS[self.enemy_index]
        self.subtitle.configure(text=f"Room: {room} | A wild {self.enemy_name} appears!")
        self.write_log(f"\nRoom {self.enemy_index + 1}: {room}")
        self.write_log(f"A wild {self.enemy_name} appears!")
        self.update_ui()

    def attack(self):
        if not self.game_active:
            return
        weapon = self.find_item("Dragon Blade") or self.find_item("Iron Sword") or self.find_item("Wooden Sword")
        damage = random.randint(10, 15) + self.weapon_level - 1 if weapon else random.randint(6, 12)
        critical = random.random() < 0.15
```

```
        weapon = self.find_item("Dragon Blade") or self.find_item("Iron Sword") or self.find_item("Wooden Sword")
        damage = random.randint(10, 15) + self.weapon_level - 1 if weapon else random.randint(6, 12)
        critical = random.random() < 0.15
        if critical:
            damage *= 2
            self.write_log("Critical strike!")
        self.enemy_hp -= damage
        self.score += damage
        self.write_log(f"You dealt {damage} damage.")
        if self.enemy_hp > 0:
            self.health -= self.enemy_damage
            self.write_log(f"{self.enemy_name} dealt {self.enemy_damage} damage.")
        self.resolve_turn()

    def heal(self):
        if not self.game_active:
            return
        if not self.find_item("Health Potion"):
            self.write_log("No potions left!")
            return
        self.inventory.remove("Health Potion")
        self.health = min(self.MAX_HEALTH, self.health + 20)
        self.score += 5
        self.write_log("You healed 20 HP.")
        self.update_ui()

    def run_away(self):
        if not self.game_active:
            return
        self.write_log(f"You escaped from the {self.enemy_name}!")
        self.enemy_index += 1
        self.begin_battle()

    def resolve_turn(self):
        if self.health <= 0:
            self.finish_game("The dungeon defeated you. GAME OVER.")
        elif self.enemy_hp <= 0:
            self.battle_history.append(self.enemy_damage)
            drop = self.collect_loot()
            self.write_log(f"You defeated the {self.enemy_name} and looted a {drop}!")
            self.upgrade_weapon()
            self.enemy_index += 1
            self.begin_battle()
        else:
            self.update_ui()

    def finish_game(self, message):
        self.game_active = False
        ranked_history = sorted(self.battle_history, reverse=True)
        self.write_log(f"\n{message}")
        self.write_log(f"Final score: {self.score} | Gold: {self.gold}")
```

```
        self.write_log(f"Final score: {self.score} | Gold: {self.gold}")
        self.write_log(f"Damage ranking: {ranked_history}")
        self.write_log(f"Inventory ({len(self.inventory)} items): {', '.join(self.inventory)}")
        self.update_ui()
        messagebox.showinfo("Dungeon Escape", message)

    def collect_loot(self):
        drop = random.choice(self.LOOT_POOL)
        self.inventory.append(drop)
        self.loot_history.append(drop)
        self.score += 50
        rewards = {"Gold Coin": 10, "Magic Scroll": 15, "Health Potion": 5}
        reward = rewards.get(drop, 0)
        self.score += reward
        if drop == "Gold Coin":
            self.gold += 10
        return drop

    def update_ui(self):
        self.player_label.configure(text=f"{self.name}: {max(0, self.health)} HP")
        self.enemy_label.configure(text=f"{self.enemy_name}: {max(0, self.enemy_hp)} HP")
        maximum = self.ENEMIES[self.enemy_index][1] if self.enemy_index < len(self.ENEMIES) else 1
        self.enemy_bar.configure(maximum=maximum, value=max(0, self.enemy_hp))
        self.draw_scene()
        self.inventory_label.configure(text=self.inventory_text())
        state = "normal" if self.game_active else "disabled"
        self.attack_button.configure(state=state)
        self.heal_button.configure(state=state)
        self.run_button.configure(state=state)

    def inventory_text(self):
        items = "\n".join(f"- {item}" for item in self.inventory) or "(empty)"
        return f"Score: {self.score}\nGold: {self.gold}\nWeapon level: {self.weapon_level}\n\n{items}"

    def find_item(self, item_name):
        """Search the inventory list and return the matching item, if found."""
        for item in self.inventory:
            if item == item_name:
                return item
        return None

    def upgrade_weapon(self):
        if self.enemy_index == 2 and self.weapon_level == 1:
            self.inventory.append(self.WEAPONS[1][0])
            self.weapon_level = self.WEAPONS[1][1]
            self.write_log("Upgrade found: Iron Sword!")
        elif self.enemy_index == 4 and self.weapon_level == 2:
            self.inventory.append(self.WEAPONS[2][0])
            self.weapon_level = self.WEAPONS[2][1]
            self.write_log("Upgrade found: Dragon Blade!")

```

```
250
251     def draw_scene(self):
252         self.scene.delete("all")
253         width = max(self.scene.winfo_width(), 600)
254         self.scene.create_rectangle(0, 0, width, 190, fill="#161d31", outline="")
255         for x in range(0, width, 48):
256             self.scene.create_line(x, 0, x, 190, fill="#202a43")
257         for y in range(0, 190, 48):
258             self.scene.create_line(0, y, width, y, fill="#202a43")
259         self.scene.create_text(115, 22, text="HERO", fill="#94a3b8", font=("Segoe UI", 10, "bold"))
260         self.scene.create_text(width - 115, 22, text=self.enemy_name.upper(), fill="#94a3b8", font=("Segoe UI", 10, "bold"))
261         self.draw_character(115, 96, "#4c9de6", False)
262         self.draw_character(width - 115, 96, "#dc4e4e", True)
263
264     def draw_character(self, x, y, color, enemy):
265         self.scene.create_oval(x - 21, y - 48, x + 21, y - 6, fill=color, outline="#f8fafc", width=2)
266         self.scene.create_rectangle(x - 28, y - 4, x + 28, y + 55, fill=color, outline="#f8fafc", width=2)
267         if enemy:
268             self.scene.create_oval(x - 10, y - 36, x - 4, y - 30, fill="#111827", outline="")
269             self.scene.create_oval(x + 4, y - 36, x + 10, y - 30, fill="#111827", outline="")
270         else:
271             self.scene.create_line(x - 38, y + 8, x + 38, y + 8, fill="#efb848", width=6)
272         self.scene.create_text(x, y + 72, text=self.name if not enemy else self.enemy_name, fill="#f8fafc", font=("Segoe UI", 10, "bold"))
273
274     def write_log(self, message):
275         self.log.configure(state="normal")
276         self.log.insert("end", message + "\n")
277         self.log.see("end")
278         self.log.configure(state="disabled")
279
280
281 def main():
282     root = tk.Tk()
283     DungeonEscapeApp(root)
284     root.mainloop()
285
286
287 if __name__ == "__main__":
288     main()
289
```